

Concepts

Visualization

NautilusTrader provides interactive HTML tearsheets for analyzing backtest results through an extensible visualization system built on Plotly. You can generate reports with minimal code and add custom charts and themes.

Overview

The visualization system has three parts:

1. **Chart Registry** - Decoupled chart definitions that can be extended with custom visualizations.
2. **Theme System** - Consistent styling with built-in and custom themes.
3. **Configuration** - Declarative specification of what to render and how to display it.

All visualization outputs are self-contained HTML files that can be viewed in any modern browser, shared with stakeholders, or archived for future reference.

 The visualization system requires `plotly>=6.3.1`. Install it with:

```
uv pip install "nautilus_trader[visualization]"
```



or

```
uv pip install "plotly>=6.3.1"
```



Tearsheets

A tearsheet is a performance report that combines multiple charts and statistics into a single interactive visualization. Tearsheets are generated after completing a backtest run and provide immediate visual feedback on strategy performance.

Quick start

Generate a tearsheet with default settings:

```
from nautilus_trader.analysis import create_tearsheet
from nautilus_trader.backtest.engine import BacktestEngine

# After running your backtest
engine.run()

# Generate tearsheet
create_tearsheet(
    engine=engine,
    output_path="backtest_results.html",
)
```

This produces an HTML file with all default charts, using the light theme and automatic layout. Open `backtest_results.html` in your browser to view the interactive tearsheet.

Customization

Control which charts appear and how they're styled:

```
from nautilus_trader.analysis import TearsheetConfig
from nautilus_trader.analysis import TearsheetDrawdownChart
from nautilus_trader.analysis import TearsheetEquityChart
from nautilus_trader.analysis import TearsheetRunInfoChart
from nautilus_trader.analysis import TearsheetStatsTableChart

config = TearsheetConfig(
    charts=[
        TearsheetRunInfoChart(),
        TearsheetStatsTableChart(),
        TearsheetEquityChart(),
        TearsheetDrawdownChart(),
    ],
    theme="nautilus_dark",
    height=2000,
)

create_tearsheet(
    engine=engine,
    output_path="custom_tearsheet.html",
)
```

```
config=config,  
)
```

Currency filtering

For multi-currency backtests, filter statistics to a specific currency:

```
from nautilus_trader.model.currencies import USD  
  
create_tearsheet(  
    engine=engine,  
    output_path="usd_only.html",  
    currency=USD, # Currency object, shows only USD statistics  
)
```

When `currency` is `None` (default), statistics for all currencies are displayed separately in the tearsheet.

Available charts

The tearsheet can include any combination of the following built-in charts:

Chart Name	Type	Description
<code>run_info</code>	Table	Run metadata and account balances.
<code>stats_table</code>	Table	Performance statistics (PnL, returns, general metrics).
<code>equity</code>	Line	Cumulative returns over time with optional benchmark.
<code>drawdown</code>	Area	Drawdown percentage from peak equity.
<code>monthly_returns</code>	Heatmap	Monthly portfolio return percentages organized by year.
<code>distribution</code>	Histogram	Distribution of individual return values.
<code>rolling_sharpe</code>	Line	60-day rolling Sharpe ratio.
<code>yearly_returns</code>	Bar	Annual return percentages.
<code>bars_with_fills</code>	Candlestick	Price bars (OHLC) with order fills overlaid as markers.

All charts are registered in the chart registry and are configured via chart objects in `TearsheetConfig.charts` (each chart object maps to a built-in chart name).

Run information table

The `run_info` chart displays key metadata about the backtest run:

- Run ID, start time, finish time
- Backtest period (start/end dates)
- Total iterations processed
- Event, order, and position counts
- Account starting and ending balances (per currency)

This table appears in the top-left position by default.

Performance statistics table

The `stats_table` chart displays performance metrics organized into sections:

- **PnL Statistics** (per currency): Total PnL, win rate, profit factor, etc.
- **Returns Statistics**: Sharpe ratio, Sortino ratio, max drawdown, etc.
- **General Statistics**: Total trades, average trade duration, etc.

This table appears in the top-right position by default.

Equity curve

The `equity` chart plots cumulative returns over the backtest period. When `benchmark_returns` is provided to `create_tearsheet()`, the benchmark is overlaid for comparison.

```
import pandas as pd

# Load benchmark returns (e.g., from a market index)
# Index should be datetime, aligned with strategy returns timeframe
benchmark_returns = pd.read_csv("sp500_returns.csv", index_col=0, parse_dates=True)["return"]

create_tearsheet(
    engine=engine,
    output_path="with_benchmark.html",
    benchmark_returns=benchmark_returns,
```

```
benchmark_name="S&P 500",
)
```

The benchmark series is plotted as-is; ensure the index aligns with your strategy's return dates for accurate comparison.

Themes

Themes control the visual styling of charts including colors, fonts, and backgrounds. NautilusTrader provides four built-in themes:

Theme Name	Description	Use Case
plotly_white	Clean light theme with dark gray headers.	Default, professional reports.
plotly_dark	Dark background with standard Plotly colors.	Low-light environments.
nautilus	Light theme with NautilusTrader brand colors.	Official light mode.
nautilus_dark	Dark theme with teal/cyan signature colors.	Official dark mode.

Selecting a theme

Specify the theme in `TearsheetConfig`:

```
config = TearsheetConfig(theme="nautilus_dark")
create_tearsheet(engine=engine, config=config)
```



Custom themes

Register a custom theme for consistent branding across all visualizations:

```
from nautilus_trader.analysis import register_theme

register_theme(
    name="corporate",
    template="plotly_white", # Base Plotly template
    colors={
        "primary": "#003366", # Navy blue
        "positive": "#2e8b57", # Sea green
        "negative": "#c41e3a", # Cardinal red
    }
)
```



```

        "neutral": "#808080",      # Gray
        "background": "#ffffff",   # White
        "grid": "#e5e5e5",         # Light gray
        # Optional table colors (defaults will be provided if omitted)
        "table_section": "#e5e5e5",
        "table_row_odd": "#f8f8f8",
        "table_row_even": "#ffffff",
        "table_text": "#000000",
    }
)

# Use the custom theme
config = TearsheetConfig(theme="corporate")

```

The theme system automatically provides sensible defaults for `table_*` colors based on the `background` and `grid` colors, ensuring backward compatibility with themes registered before table-specific colors were introduced.

Configuration

The `TearsheetConfig` class provides declarative control over tearsheet generation:

```

from nautilus_trader.analysis import GridLayout
from nautilus_trader.analysis import TearsheetConfig
from nautilus_trader.analysis import TearsheetDrawdownChart
from nautilus_trader.analysis import TearsheetEquityChart
from nautilus_trader.analysis import TearsheetStatsTableChart

config = TearsheetConfig(
    charts=[
        TearsheetEquityChart(),
        TearsheetDrawdownChart(),
        TearsheetStatsTableChart(),
    ],
    theme="nautilus_dark",
    title="Q4 2024 Strategy Performance",
    height=1800,
    include_benchmark=True,
    benchmark_name="SPY",
    layout=GridLayout(
        rows=2,
        cols=2,
        heights=[0.60, 0.40],
        vertical_spacing=0.08,
        horizontal_spacing=0.12,
    ),
)

```



Configuration parameters

Parameter	Type	Default	Description
<code>charts</code>	<code>list[TearsheetChart]</code>	All built-in charts	List of chart objects to include (in order).
<code>theme</code>	<code>str</code>	<code>"plotly_white"</code>	Theme name for styling.
<code>layout</code>	<code>GridLayout</code>	<code>None</code> (auto-calculated)	Custom subplot grid layout.
<code>title</code>	<code>str</code>	Auto-generated with strategy/time	Tearsheet title.
<code>include_benchmark</code>	<code>bool</code>	<code>True</code>	Show benchmark when provided.
<code>benchmark_name</code>	<code>str</code>	<code>"Benchmark"</code>	Display name for benchmark.
<code>height</code>	<code>int</code>	<code>1500</code>	Total height in pixels.
<code>show_logo</code>	<code>bool</code>	<code>True</code>	Display NautilusTrader logo (reserved for future use).

When `layout` is `None`, the grid dimensions and row heights are automatically calculated based on the number of charts. For 8 charts (the default), a 4×2 grid is used with heights `[0.50, 0.22, 0.16, 0.12]` to give more space to the top row tables.

Custom charts

The registry pattern lets you add custom charts. Charts are functions that render traces onto a Plotly figure object.

Registering a custom chart

```
from nautilus_trader.analysis.tearsheet import register_chart
import plotly.graph_objects as go

def my_custom_chart(returns, output_path=None, title="Custom Chart", theme="plotly_white"):
    """
```



Create a custom visualization.

This function signature matches the built-in chart functions for consistency.

"""

```
from nautilus_trader.analysis.themes import get_theme
```

```
theme_config = get_theme(theme)
```

```
# Create your visualization
```

```
fig = go.Figure()
```

```
fig.add_trace(go.Scatter(
    x=returns.index,
    y=returns.cumsum(),
    mode="lines",
    name="Custom Metric",
    line={"color": theme_config["colors"]["primary"]},
))
```

```
fig.update_layout(
    title=title,
    template=theme_config["template"],
    xaxis_title="Date",
    yaxis_title="Value",
)
```

```
if output path:
```

Tearsheet integration

For full tearsheet integration with proper grid placement, use the lower-level registration.

⚠ The `_register_tearsheet_chart` function is internal API and may change between releases. For most use cases, prefer `register_chart` for standalone charts or contribute new built-in charts upstream.

```
from nautilus_trader.analysis import TearsheetConfig
from nautilus_trader.analysis import TearsheetCustomChart
from nautilus_trader.analysis import TearsheetEquityChart
from nautilus_trader.analysis import TearsheetStatsTableChart
from nautilus_trader.analysis.tearsheet import _register_tearsheet_chart
```

```
def _render_my_metric(fig, row, col, returns, theme_config, **kwargs):
```

"""

Render custom metric directly onto a subplot.

Parameters

fig : go.Figure

The figure to add traces to.

row : int

Subplot row position.

col : int


```

        Subplot column position.
    returns : pd.Series
        Strategy returns from analyzer.
    theme_config : dict
        Theme configuration dictionary.
    **kwargs : dict
        Additional parameters (stats_pnls, stats_returns, benchmark_returns, etc.).
    """
    metric_values = returns.rolling(30).std() * 100 # Example metric

    fig.add_trace(
        go.Scatter(
            x=returns.index,
            y=metric_values,

```

The renderer function receives all necessary data (returns, statistics, theme configuration) and renders directly onto the specified subplot position.

Offline analysis

For situations where you have precomputed statistics but not a `BacktestEngine` instance, use the lower-level API:

```

from nautilus_trader.analysis.tearsheet import create_tearsheet_from_stats

# Load precomputed data (structure matches PortfolioAnalyzer output)
stats_pnls = {"USD": {"PnL (total)": 1500.0, "Win Rate": 0.55, ...}} # Per-currency
stats_returns = {"Sharpe Ratio (252 days)": 1.2, "Max Drawdown": -0.15, ...}
stats_general = {"Avg Winner": 100.0, "Avg Loser": -50.0, ...}
returns = pd.Series(...) # Daily returns with datetime index

create_tearsheet_from_stats(
    stats_pnls=stats_pnls,
    stats_returns=stats_returns,
    stats_general=stats_general,
    returns=returns,
    output_path="offline_analysis.html",
)

```

The dictionary keys should match those returned by `PortfolioAnalyzer.get_performance_stats_*()`.

This approach is useful for:

- Analyzing results from multiple backtest runs stored separately.
- Comparing strategies using precomputed metrics.

- Integrating with external analysis pipelines.

Best practices

Chart selection

- Use default charts for exploratory analysis to see all available metrics.
- Customize charts when you know which metrics matter for your strategy.
- Remove irrelevant charts to reduce visual clutter and file size.

Theme usage

- Use `plotly_white` for professional reports and presentations.
- Use `nautilus_dark` for official materials or low-light viewing.
- Create custom themes to match internal guidelines or personal preferences.

Performance considerations

- Tearsheet HTML files contain all data inline and can be several megabytes for long backtests.
- Consider generating separate tearsheets for different analysis timeframes.
- For very large datasets, use the individual chart functions instead of full tearsheets.

Custom statistics integration

Custom charts work best when paired with [custom statistics](#) registered in the `PortfolioAnalyzer`. This ensures your visualizations display metrics computed consistently with the rest of the system:

```
from nautilus_trader.analysis.statistic import PortfolioStatistic

class MyCustomStatistic(PortfolioStatistic):
    """Custom metric for specialized strategy analysis."""

    def calculate_from_returns(self, returns):
        # Your calculation logic
        return custom_metric_value

# Register with analyzer
```



```
analyzer.register_statistic(MyCustomStatistic())  
  
# Now available in stats_returns for custom charts
```

API levels

The visualization system provides two API levels:

High-level API

Recommended for most use cases:

```
create_tearsheet(engine=engine, config=config)
```



Automatically extracts data from the `BacktestEngine`, generates all configured charts, and produces a complete HTML tearsheet.

Low-level API

For advanced customization or offline analysis:

```
create_tearsheet_from_stats(  
    stats_pnls=stats_pnls,  
    stats_returns=stats_returns,  
    stats_general=stats_general,  
    returns=returns,  
    run_info=run_info,  
    account_info=account_info,  
    config=config,  
)
```



Provides fine-grained control over data inputs and allows analysis of precomputed statistics.

Standalone chart functions

Individual chart functions can be used independently to generate single-purpose HTML visualizations or Plotly figures for custom analysis workflows.

Price bars with fills

The `create_bars_with_fills` function generates a candlestick chart with order fills overlaid, useful for visually analyzing strategy execution within price action. It can be used standalone or included in tearsheets:

```
from nautilus_trader.analysis import create_bars_with_fills
from nautilus_trader.analysis import create_tearsheet
from nautilus_trader.analysis import TearsheetBarsWithFillsChart
from nautilus_trader.analysis import TearsheetConfig
from nautilus_trader.analysis import TearsheetEquityChart
from nautilus_trader.analysis import TearsheetStatsTableChart
from nautilus_trader.model.data import BarType

# Standalone usage
bar_type = BarType.from_str("ESM4.XCME-1-MINUTE-LAST-EXTERNAL")
fig = create_bars_with_fills(
    engine=engine,
    bar_type=bar_type,
    title="ES Futures - Entry/Exit Analysis",
)
fig.show() # Display in Jupyter
fig.write_html("bars_with_fills.html") # Or save to file

# Include in tearsheet
config = TearsheetConfig(
    charts=[
        TearsheetStatsTableChart(),
        TearsheetEquityChart(),
        TearsheetBarsWithFillsChart(
            bar_type="ESM4.XCME-1-MINUTE-LAST-EXTERNAL",
            title="Bars with Fills",
        ),
    ],
)
create_tearsheet(engine=engine, config=config)

# Multiple bars-with-fills charts in one tearsheet
```

The visualization shows candlesticks for OHLC price action with triangle markers representing order fills (green up-triangles for buys, red down-triangles for sells). Charts that need extra configuration (like `bar_type`) take those parameters directly on the chart object (e.g.

`TearsheetBarsWithFillsChart(bar_type=...)`).

Other individual chart functions include `create_equity_curve`, `create_drawdown_chart`, `create_monthly_returns_heatmap`, and more. See the API reference for the complete list.

Related guides

- [Backtesting](#) - Learn how to run backtests that generate tearsheets.
- [Reports](#) - Understand the underlying statistics displayed in tearsheets.
- [Portfolio](#) - Explore portfolio tracking and performance metrics.

[< Backtesting](#)[Previous Page](#)[Configuration >](#)[Next Page](#)